B3 Clean pandas

Dtypes, vectorisation, method chaining and Copy-on-Write in pandas 3.

B3-L1 Dtypes in pandas 3, nullable and string

Decide dtypes at load time. IDs and codes as str, counts with gaps as Int64.

```
df = pd.read_csv(
    path,
    dtype={"respondent_id": "str", "postcode": "str", "household_size": "Int64"},
)
```

Default text dtype in pandas 3.0	str, with NaN for missing values. Only strings are allowed.
int64 vs Int64	int64 is NumPy, no missing values. Int64 is the pandas nullable integer that holds pd.NA.
Keep leading zeros in IDs	Read the column with dtype str in read_csv.
Assign 1.5 into an int64 Series on pandas 3	Raises TypeError instead of silently upcasting.
Where to set dtypes	At load time, from a dtype map in config. Check df.dtypes right after.

B3-L2 Vectorise instead of apply

The strong version is shorter, about 500 times faster on large frames, and handles missing ages on purpose.

Why is df.apply(func, axis=1) slow?	It calls a Python function once per row. np.where measured about 500 times faster on one million rows.
Vectorised if-else in NumPy	np.where(cond, value_if_true, value_if_false). For several conditions, np.select.
What does NaN > 60 return?	False. np.where and np.select send missing values to a real category unless you handle them.
Numeric bands that keep missing values missing	pd.cut(values, bins, labels, right=False).

B3-L3 Method chaining and Copy-on-Write

Under Copy-on-Write, write with .loc in one step, or better, build a new frame with a chain of assign, loc and pipe.

```
df[df["age"] < 18]["weight"] = 0       # df unchanged, ChainedAssignmentError warning
df.loc[df["age"] < 18, "weight"] = 0   # one step: df changed
```

```
result = (
    raw
    .rename(columns=COLUMN_MAP)
    .loc[lambda d: d["age"] >= 18]
    .assign(weight=lambda d: d["weight"].clip(upper=5.0))
    .pipe(add_age_band, bins=BINS, labels=LABELS)
)
```

Copy-on-Write in pandas 3.0	Always on. Indexing results behave as separate objects, copied only on write.
df[mask]["col"] = value on pandas 3	Changes nothing and emits ChainedAssignmentError. Use df.loc[mask, "col"] = value.

Refer to the current step inside a chain	A lambda: assign(x=lambda d: ...) or loc[lambda d: ...].
Call your own function inside a chain	.pipe(func, **kwargs).